



PHOTON64

Remaining-defect audit and implementation plan

Code-grounded review of the emulator, browser app and validation pipeline.

10 October 2026 | Source 96a7f073dffa8cf002b3e5bb83f20ce37da66d3

18 findings | 10 reproduced defect classes | 9 implementation phases

Primary priorities: save integrity, GPU ownership and trustworthy browser evidence.

Review status

The latest exact-source CI run passed seven jobs and failed native Safari. This report includes targeted reproductions, confidence levels and acceptance criteria. Production code was not changed by this audit.

Contents

1. Executive assessment
2. Evidence and coverage
3. Findings F01-F18
4. Step-by-step implementation plan
5. Acceptance and delivery discipline
6. Reproduction guide
7. Source references

Review date: 10 October 2026. **Audited source:** 96a7f073dffaa8cf002b3e5bb83f20ce37da66d3 on main.
Scope: C/WASM core, RSP and peripheral paths, RDP/VI and WebGPU coherence, browser UI, persistence, input/audio, build/test/release workflows, and project documentation.

This is a new code-grounded audit. Earlier reports describe completed milestones and historical failures; they are not substituted for examination of the current implementation. No production fixes are included in this audit. The accompanying probes observe the audited implementation and deliberately expose defects that the existing tests do not detect.

1. Executive assessment

Photon64 has a substantial foundation: a compact standalone app, two rendering paths, independent native renderer comparisons, carefully bounded imports, transactional save writes, and broad software-renderer browser coverage. The earlier work fixed real problems and should be retained. The remaining issues are concentrated at boundaries: persistence across machine restoration, CPU address faults, circular GPU memory ownership, asynchronous renderer changes, and the difference between tested software behavior and actual mobile WebGPU execution.

Recommendation: keep the project at preview status. Prioritize save integrity and GPU failure diagnosis before another accuracy or performance expansion. Do not call the latest revision fully validated: its native Safari job currently fails, despite the other seven jobs passing.

The review reproduced **ten defect classes**: reset losing dirty-save status, state restore leaving durable battery data stale, wrapped GPU ranges being truncated, obsolete scale failures overriding newer requests, partial GPU allocations lacking deterministic cleanup, cartridge identity collisions, file-selection order inversion, incorrect merge-load fault addresses, missing privileged-segment enforcement, and signed-shift undefined behavior. Additional source-confirmed omissions and test gaps are described separately. There is no evidence here of a browser sandbox escape or of every supported game being affected.

Priority and confidence definitions

- **P1:** protect progress, prevent rendering/coherence failures, or restore trustworthy release evidence. Address before promoting a new preview as broadly validated.
- **P2:** correctness, compatibility, resource management or maintainability work for subsequent milestones.
- **P3:** product improvements after the foundation is reliable.
- **Reproduced:** an included probe executes production functions and demonstrates the stated behavior. A mocked GPU probe proves host control flow, not physical GPU behavior.
- **Source-confirmed:** directly visible implementation omission or architectural constraint; consequences may depend on a workload not exercised here.
- **Observed / unresolved:** actual failure evidence exists but the cause is not established.

2. Evidence and coverage

Current validation state

[GitHub Actions run 38081746008](#), attempt 1, validates the exact audited SHA:

Job	Result	What the result supports
Baseline	PASS	Existing native/WASM/frontend/reference gates
Chromium desktop and mobile emulation	PASS	Browser software-renderer lifecycle coverage
Branded Chrome and Edge	PASS	Desktop software-renderer lifecycle and responsive UI coverage
Playwright WebKit desktop and mobile emulation	PASS	WebKit software-renderer lifecycle and responsive UI coverage
Native macOS Safari	FAIL	Narrow and landscape completed; later desktop sequence timed out

Native Safari also reports **SKIP** for the independent OS-backed file read: `NotReadableError` before the app participates. Browser-backed File injection remains a distinct exercised path. The failed job is [114300299589](#). Its later timeout yielded `failureState: null`; the logs do not establish an app-versus-driver cause. A successful retry of the previous revision does not make this revision green.

Fresh local checks in this audit:

- `npm test`: 15 legacy JS checks and 42 Node tests pass.
- Native diagnostic probe: wrong fault addresses, user-mode kernel read, truncated wrapped stale range, and reset dirty-state loss reproduced.
- UBSan: two separate high-bit byte assembly paths fail with signed-left-shift diagnostics.
- Frontend probe: real WASM plus existing mocked browser/IndexedDB scaffolding reproduces reset, restore, identity and file-order defects; mocked GPU control flow reproduces range, scale and allocation issues.
- Runtime allocation inspection: initial WASM memory 58,982,400 bytes; static-state snapshot 58,940,224 bytes; after the reusable ROM reserve, 134,479,872 bytes. Built HTML is 434,733 bytes; WASM is 155,821 bytes. Download size and runtime memory are very different budgets.

The full baseline was not rerun locally because the exact-revision hosted baseline had already passed. No new commercial gameplay, physical-phone, controller or audio-device campaign was executed. Local Chromium was not launched, following `AGENTS.md`'s environment restriction. Historical commercial replay counts remain historical evidence, not freshly repeated results.

What existing renderer evidence does and does not mean

`VALIDATION_REPORT.md` records five native reference replays with zero counted differences after the previous fixes. Those comparisons use a pinned guest-write/noise-zero reference adapter. Gameplay GPU checks use selected six-field windows around checkpoints, last-batch targets and a small set of execution counters. Authored GPU streams additionally compare twelve retained registers. These are valuable bounded checks, but neither full-game compatibility nor continuous GPU execution nor console-hardware certification follows from them.

The current workflow does not invoke `tools/device_loss.mjs`, `tools/rdp_gpu_vectors.js`, or the recorded GPU gameplay runner. All browser matrix harnesses explicitly select software rendering. This distinction is central to findings F11-F13.

3. Findings

F01 - Reset can discard the obligation to persist battery progress

P1 | Reproduced with real WASM. Locations: `src/web/app.js:resetGameNow` (1090), `src/bus.c:sys_reset` (551), `src/web/app.js:flushSaves` (354).

`sys_reset()` clears `sys`, including `save_dirty`, while preserving the separate EEPROM, SRAM/Flash and Controller Pak arrays. `resetGameNow()` neither waits for a durable battery commit nor preserves the generation. The probe writes byte `0xA5`, marks generation 7, resets, and flushes: RAM still contains `0xA5`, `dirty` is zero, and no battery record exists.

Opening the menu normally starts a flush, which reduces the ordinary exposure but does not establish a reset barrier. A failed or unfinished write, or another authorized reset path, can still reach this condition. Continued play can look correct until reload loses the uncommitted progress.

Recommendation: make battery preservation an explicit reset invariant. Preserve dirty generations across soft reset and/or require a successful flush before clearing machine state. Distinguish ordinary reset, imported-save reset and cartridge replacement so one path cannot accidentally save over another. Keep an export route usable when storage fails.

Acceptance: reset immediately after a guest save, during a held IndexedDB transaction, after quota failure and in temporary mode. After reload, bytes must match the last promised durable save; failures must retain dirty bytes and report them.

F02 - Loading a state can restore battery bytes without updating persistence

P1 | Reproduced with real WASM. Locations: `src/web/app.js:saveStateNow` (390), `loadStateNow` (411), `flushSaves`.

The state copies all static memory, including battery bytes and the old dirty generation. A state saved when the battery was clean later restores that zero generation. Probe: persist `0x11`, save state, persist `0x22`, load the earlier state, flush. Memory becomes `0x11`, durable storage stays `0x22`, `dirty` is zero. The next reopen silently restores the newer battery instead of the loaded machine's battery.

An in-flight autosave can also outlive a restore. Existing identity/generation checks stop an old callback from clearing the new dirty flag, but they do not alone define which durable battery image should win.

Recommendation: explicitly define state/battery policy. The recommended default is that loading a state restores its battery memory and creates a fresh dirty generation independent of snapshot bookkeeping. Drain prior save writes before mutation; keep host persistence ownership outside restorable guest memory. If retaining the current durable battery is desired instead, exclude it from restore and clearly expose that policy.

Acceptance: the `0x11` -> `0x22` -> restore sequence must have one documented durable outcome after reload. Repeat with delayed commits, multiple state loads, failed writes and no subsequent guest save activity.

F03 - GPU ownership and synchronization truncate circular RDRAM ranges

P1 | Reproduced in native tracking and JavaScript range handling. Locations: `src/gpu.c:gpu_mark_stale` (42), `gpu_watch_region` (81); `src/web/gpu.js:syncRange` (235), `touch` (307); `src/web/rdp.wgsl:px_load/px_store` (691/717).

Shader addressing wraps the physical framebuffer index. Host and core tracking clamp at the end of the 8 MiB RDRAM address space. For a four-halfword interval beginning at halfword 4,194,302, the host

uploads/tracks only the final two halfwords; the required $[0, 2)$ interval disappears. Native stale tracking similarly marks the end but not address zero. VI apron reads near address zero also exercise negative-start handling that currently clips rather than wraps.

This can let CPU reads observe old bytes or CPU writes bypass watched-page handling even though GPU work touched the wrapped memory. The probe establishes the tracking mismatch; no commercial-game visible failure has been attributed to it in this audit.

Recommendation: define one circular-interval contract and implement equivalent split logic in C and JS for upload, watch, stale, touched/readback and VI source ranges. Normalize addresses, handle zero length, cap full-memory coverage, and preserve job exclusions. Do not fix only `touch()`.

Acceptance: render across the RDRAM boundary in RGBA16/32 and depth; verify wrapped bytes and hidden bits, CPU load/store/DMA barriers, overlapping snapshots, and negative VI apron ranges. Execute real GPU vectors at native and full-storage HD-at-1x against independently checked expectations.

F04 - An obsolete scale failure overrides a newer successful selection

P1 | Reproduced with production `applyScale` and controlled promises. Locations: `src/web/app.js:applyScale (89)`; `src/web/gpu.js:setScale (157)`.

The lower layer has a sequence number, but the frontend catch handler unconditionally sets the desired scale and settings to zero and calls `setScale(0)`. Start 2x preparation, select 4x, let 4x succeed, then reject the older 2x request: observed calls are `[1, 2, 0]` and final settings return to native.

Recommendation: assign a frontend request generation and renderer identity to scale changes; stale success and failure callbacks must not change current UI/settings. Serialize mutation of the active rendering set, while allowing preparation to finish harmlessly. Include renderer switching, reset and loss in the same ownership rules. Avoid overlapping error-scope stacks around asynchronous builds.

Acceptance: rapid 1x/2x/4x changes in every resolve/reject order; software switch, reset and loss during build. Latest valid request wins, with no stale toast, unwanted fallback or destroyed active resource.

F05 - Partial GPU builds lack deterministic resource cleanup

P2 | Reproduced with a mocked device and production `buildSet`. Locations: `src/web/gpu.js:buildSet (60)`, `setScale`, `readBuf (457)`, `readOutput (470)`.

`buildSet()` allocates its target, two textures and feedback buffer before awaiting further pipelines. If a later pipeline fails, the promise rejects before the caller receives `set`, so its cleanup cannot destroy those allocations. The probe records four allocations and zero explicit destroys. This proves missing deterministic cleanup, not permanent leakage: eventual browser garbage collection may reclaim objects. Readback helper buffers likewise lack `finally` cleanup after rejected mapping.

Recommendation: use a resource owner during construction; transfer ownership only after every pipeline and bind group succeeds. Destroy all partial resources on rejection or cancellation. Add a renderer disposal method and `try/finally` to temporary readbacks.

Acceptance: fail each build stage and map stage once; all allocated disposable resources are destroyed exactly once. Repeat failed 4x attempts under memory pressure and show stable resource counts and recovery to a valid set.

F06 - Header-only cartridge identity can load the wrong cached image

P1 | Reproduced with two synthetic cartridges. Locations: `src/web/app.js:romIdentity (217)`, `libAdd (1131)`, `loadStateNow`.

The key is title plus the eight header CRC bytes; it is not a digest of the bytes actually loaded. `libAdd()` does not replace an existing ROM under that key. Two same-size images with identical headers and a changed payload produce one library entry: the active image has the new byte while cached reopening retains the original. Patches/homebrew with unchanged headers are exposed; this does not claim collisions among all correctly checksummed retail dumps.

Battery records and states share the key, and the state header adds core hash and ROM size but no content digest. Consequently the identity ambiguity extends beyond the library label.

Recommendation: hash canonical byte-order ROM contents with SHA-256. Store display metadata separately. Distinguish cartridge identity from an explicitly chosen save-sharing group. Migrate legacy entries transactionally and retain backup records; do not silently erase existing saves or automatically merge patched games.

Acceptance: equivalent z64/n64/v64 representations share identity; a one-byte payload change does not. Same-header variants survive library reopen and cannot cross-load states. Legacy saves remain recoverable through a documented migration.

F07 - File reads happen before the session queue and invert selection order

P2 | Reproduced. Locations: `src/web/app.js:openFile` (203), `loadRom` (212), file/drop handlers.

The session queue orders `loadRom()` calls, but `openFile()` first awaits reading and decompression. Select slow file A, then fast file B: B loads first, then A unexpectedly replaces it. The probe ends with the first-selected cartridge active. Concurrent selections also retain multiple large candidate buffers.

Recommendation: define selection semantics, preferably latest selection wins. Assign a token at the user event before any await, cancel obsolete decoding where possible and reject stale completion before allocation or persistence mutation. Keep session operations serialized after selection admission.

Acceptance: A slow/B fast, ZIP/raw, failed newer file, library click during file read, home during decompression, and repeated 64 MiB inputs. The documented selection wins and stale work cannot reset it.

F08 - Merge loads report the aligned address on TLB faults

P2 | Reproduced for LDL/LDR/LWL/LWR. Locations: `src/cpu.c` opcode cases 26, 27, 34 and 38; `rd64u`, `rd32u`, `tlb_exception`.

These opcodes align the virtual address before calling a helper that performs translation. The original unaligned address is therefore unavailable to the exception path. All four probes at virtual address `0x4003` report `BadVAddr=0x4000` with TLBL. Earlier store-side repairs do not cover this load-side case.

Recommendation: separate the original fault address from the aligned physical fetch. Preserve the original address when entering TLB exceptions without altering merge semantics or retry behavior. NEC documents the faulting virtual address and delay-slot exception bookkeeping [S1].

Acceptance: every byte alignment, miss/invalid TLB, ASID change, branch delay slot and GPU retry, with unchanged destination on fault. Execute the same independent vectors in native sanitizers and WASM O0/O3.

F09 - User mode can access kernel direct-mapped memory

P2 | Reproduced. Locations: `src/cpu.c:translate` (143), fast map accesses and EA; `cpu_reset` map setup.

The probe executes from a mapped user page with KSU=user and EXL/ERL clear, then loads from `0x80002000`. It reads `0x12345678` with no exception. Both the fast maps and direct translation path lack

the required segment privilege check. This is guest CPU conformance, not isolation between the emulator and its host browser.

Recommendation: make segment permissions part of both fast and slow access paths, including fetch. Treat 32-bit privilege enforcement as a bounded first change; separately plan full 64-bit virtual address/PC support. NEC defines distinct user, supervisor and kernel spaces [S1].

Acceptance: matrix of KSU, EXL, ERL and segment for fetch/load/store; unchanged RAM/register state on fault; correct exception address. Follow with UX/SX/KX canonical-address and 64-bit opcode coverage rather than claiming that one patch completes MIPS III conformance.

F10 - Native byte assembly still contains signed-shift undefined behavior

P2 | Reproduced under UBSan. Locations: `src/api.c:67` ROM magic; `src/bus.c:135` SRAM read assembly. Similar patterns deserve review in PIF and command packing.

The C integer promotions turn `data[0] << 24` into a signed shift. A normal big-endian N64 header starts with `0x80`, which triggers UBSan. A SRAM word with high byte `0x80` triggers the same diagnostic. Current production WASM happens to load these fixtures, but that does not make the C expression defined or validate other compilers/optimization levels.

Recommendation: cast to `u32` before the shift or centralize explicit big-endian helpers. Inspect signed shifts in RDP conversion packing as well; avoid blanket cast edits without checking arithmetic intent.

Acceptance: all three ROM byte orders, high-bit PIF/SRAM values and representative signed renderer factors under ASan/UBSan plus native/WASM O0/O3. Add actual ROM loading to sanitizer coverage; existing manually initialized CPU vectors bypass this path.

F11 - Runtime asynchronous GPU errors are not fully observed

P1 | Source-confirmed observation gap; device consequences need integration tests.

Locations: `src/web/gpu.js:init`, `flush`, `present (433)`; `src/web/app.js:rendererFailure`.

Device loss is handled, but no `uncapturederror` handler is installed. Runtime submissions generally lack scoped validation observation, and `onSubmittedWorkDone()` rejection increments the same completion counters as success. WebGPU operations can report errors asynchronously; a synchronous `try/catch` is insufficient [S2]. Existing fake-device tests mostly throw or reject explicit mapping/submission calls, which does not cover all browser error delivery.

The user's World Driver screenshot displays the generic synchronization-failure message at 4x. It confirms an incident but contains neither the original exception nor adapter/browser metadata needed to attribute it to F03, memory pressure or device loss.

Recommendation: centralize renderer fault reporting with phase, original message, device-lost reason, scale, queue counts, batch size, source/build identity and optional cartridge digest. Keep this local with an exportable diagnostic file; never include ROM bytes. Capture validation/internal/OOM events, preserve the first cause, and provide a controlled reset/software recovery path. Treat rejected completion as failure evidence.

Acceptance: actual-device invalid submission, mapping rejection, deliberate loss, failed pipeline and failed screenshot. No false success or premature CPU acknowledgment; export remains available and recovery is explicit.

F12 - Required CI does not execute GPU correctness or device-loss lanes

P1 | Source-confirmed coverage gap. Locations: `.github/workflows/validate.yml`; `tools/browsercheck.mjs`, `responsivecheck.mjs`, `safaricheck.mjs`.

The matrix exercises the app through software rendering. Useful GPU harnesses exist but are not required by the current workflow. A shader/coherence regression can therefore pass the advertised browser matrix. Historical one-off Dawn success is not an ongoing gate.

Recommendation: add a required, named Dawn/software-adapter command-vector and device-loss job using shipped or authored fixtures. Add full-storage native/HD-at-1x comparisons with failure artifacts. Run continuous GPU gameplay locally on the authorized cartridges; keep commercial bytes and traces out of CI. Add physical adapters as separately named scheduled/manual evidence when available.

Acceptance: an intentionally wrong shader result and a deliberately missing adapter each prevent an exact GPU gate from reporting PASS. Native and full-storage HD-at-1x compare memory, hidden/depth data and feedback registers. Required job names must be explicitly checked by release automation.

F13 - Native Safari reliability and direct-file behavior remain unresolved

P1 | Observed failure, cause unresolved. Locations:

tools/safaricheck.mjs:command/wait/catch/finally; current Actions run; UI_COMPATIBILITY.md.

The native job stalls after successful narrow and landscape phases. Each WebDriver request has a 90-second deadline; a nominal polling loop is not a single global deadline. Failure collection, screenshot and cleanup can then each wait again on the unresponsive session, obscuring the first failing command. The OS-backed chooser is separately skipped. All main browser harnesses serve HTTP, while the product foregrounds direct opening of the HTML file.

Recommendation: journal command/phase start and finish before awaiting; checkpoint results continuously; give diagnostics and cleanup short independent budgets. Capture driver/process state and the last successful action. Reproduce desktop alone and in the full sequence. Retain first-attempt failures even when rerunning. Add a direct-file launch/storage/audio/GPU smoke matrix and genuine iOS/iPadOS file-picker testing.

Acceptance: five clean native runs without unexplained retries as an initial stability target; any failure identifies a phase and returns bounded diagnostics. OS-backed input remains SKIP until observed working. Published browser claims name actual browser, OS, origin mode and renderer.

F14 - Broad two-cycle serialization creates a likely GPU bottleneck

P2 | Source-confirmed architecture; physical cost unmeasured here. Locations:

src/rdp.c:420-428, src/web/rdp.wgsl:ordered_main, src/web/gpu.js:flush.

Every two-cycle primitive marks the batch ordered. One compute invocation then loops over primitives, spans and pixels; 2x/4x increases display work. This preserves the chosen feedback semantics but can remove most GPU parallelism. It is a plausible contributor to slow frames or watchdog pressure, not an established explanation for the screenshot.

Recommendation: first record ordered batch counts, shaded samples, dispatch duration, maximum batch dimensions, readback latency and frame-time percentiles. Then classify true dependencies and split unrelated work into parallel batches. Bound dispatch work while carrying feedback across chunks. Never replace ordered feedback with a pixel-local approximation just to improve FPS.

Acceptance: exact native and HD-at-1x results before/after, no new physical-device losses, and a measured improvement on named workloads/adapters. Measure long frames and input/audio latency, not only average FPS.

F15 - Small download size hides substantial runtime memory demand

P2 | Measured WASM allocation; source-derived GPU budget. Locations:

src/api.c:n64_alloc, src/web/app.js:loadRom/openFile/stateMeta, src/web/gpu.js:buildSet/setScale.

The app is about 425 KiB on disk but reserves about 128.25 MiB of WASM memory after a cartridge allocation. The static state image is about 56.21 MiB. The 4x GPU target alone is 256 MiB, excluding the native 16 MiB target, textures, staging, decoded ROM arrays, library copies and compression buffers. This is not a measured whole-process peak; those resources have different lifetimes.

`stateMeta()` reads full state payloads when metadata is absent merely to determine slot existence. Overlapping file reads and old/new resolution sets further raise transient demand.

Recommendation: establish per-operation memory budgets; replace existence reads with key/count queries; reduce unnecessary copies; bound concurrent loads and readbacks; consider a growable reusable ROM allocation with transactional replacement. Default to a conservative scale, retain native fallback, and expose an accurate scale failure reason.

Acceptance: record steady/peak process and GPU memory for 8/32/64 MiB cartridges, ZIP loading, four states and scale transitions on target devices. Repeated operations reach a plateau, with intact saves after allocation failure.

F16 - Compatibility omissions remain beyond the renderer

P2 | Source-confirmed and previously documented limits. Locations: `src/bus.c:pif_control(331)`, `src/cpu.c:CACHE/EA`, `src/api.c:save_db`, `src/web/app.js:readInput`.

CIC challenge/response is explicitly acknowledged without computing a response. CACHE is a no-op, effective addresses/PCs are truncated to 32 bits, and scheduling uses constant CPI. Save hardware defaults to 4K EEPROM for unmatched IDs; automatic SRAM/Flash transitions require `SAVE_NONE`, so the default does not provide universal autodetection. Browser input sends only controller port zero and stops after the first connected gamepad, despite the core exposing four ports.

Recommendation: publish a capability matrix separating missing features from rendering defects. Implement and independently validate CIC behavior with suitable permitted expectations. Introduce auditable cartridge/save metadata plus a safe manual override and backup/export. Plan CPU timing/cache/64-bit addressing as distinct conformance projects. Treat multi-controller UI as a product feature with disconnect/reconnect and mapping tests.

Acceptance: targeted CIC protocol vectors; save-media read/write/power-cycle tests by game ID and revision; independent CPU hardware vectors; two-to-four controller port tests. Do not derive compatibility from a 300-field boot or copy implementation from an upstream with unresolved provenance.

F17 - Audio, accessibility and persistence usability need device-level work

P2/P3 | Improvement opportunities; failures not claimed without reproduction. Locations: `src/web/app.js:audioStart/Resampler/idbOpen/renderLibrary`; `src/web/app.html`.

Audio has dynamic queue control and a fallback, but no surfaced underrun/overrun metrics or automated physical interruption tests. Partial audio initialization can leave an existing context with no ready output, preventing a fresh initialization attempt. Library cards nest a remove button inside a `role=button` container; the current DOM geometry/focus checks are not a screen-reader audit. Storage writes are transactional within one page but library read-modify-write and battery updates have no cross-tab ownership policy. State slots lack an export/migration path across core builds.

Recommendation: add audio readiness/retry states, diagnostics and interruption tests; use sibling play/remove controls with unambiguous names; test VoiceOver and keyboard navigation; define one active writer per cartridge with cross-tab notification; add storage usage/backup management and portable battery metadata. Consider state export only with explicit build compatibility checks.

Acceptance: headset changes, tab/lock interruptions, autoplay refusal, 44.1/48 kHz output, sustained play; screen-reader navigation with 200% text; simultaneous tabs cannot silently overwrite progress; browser eviction and version upgrades have a documented recovery path.

F18 - Release evidence, provenance and audit navigation need tightening

P2 | Source-confirmed process gaps. Locations: `.github/workflows/release.yml`, `THIRD_PARTY.md`, historical reports, dependency manifests.

The release workflow appropriately requires a successful exact-source run and publishes tested HTML with hashes. However, its job test is a count plus baseline/Safari names, not an exact required matrix. Notices are copied from the publishing checkout rather than explicitly from the selected source SHA. Actions use major-version tags. Fixture distribution permissions and derivation/provenance review are expressly unresolved in `THIRD_PARTY.md`. This audit does not offer a legal determination.

Recommendation: verify exact expected job identities, pin notice sources to the released revision, record tool versions and artifact digests, and consider immutable action SHAs with managed updates. Resolve provenance through source records and maintainers/qualified review before broad redistribution claims. Add a current-status index linking historical reports and dated evidence; retain their original outcomes.

Acceptance: missing/renamed required jobs, wrong artifact/source and mismatched notices reject release packaging. A release manifest maps every shipped component to its recorded provenance and license notice. CI failures and SKIPS remain visible.

4. Step-by-step implementation plan

The following is a complete sequence for the findings above. Estimates are rough engineering effort for one experienced contributor, including tests, not calendar promises. Hardware access and unexplained browser failures can dominate elapsed time. Work remains on main under the project's current policy, using small reviewable commits and fast-forward publication.

Phase 0 - Preserve evidence and improve failure visibility (1-3 days)

1. Retain this audited SHA, probes and observations. Create a working issue/checklist for each finding with its confidence and expected outcome.
2. Add phase timestamps, bounded failure collection and incremental evidence writing to native Safari. Run desktop alone, then the complete sequence; preserve first-failure logs.
3. Add local renderer error capture and a diagnostic export containing build/browser/adaptor/scale and first-cause details. Keep ROM contents and save bytes out.
4. Reproduce World Driver at native, 2x and 4x on the user's affected device using the same input/checkpoint, recording exact failure phase. Until this is available, keep the incident unassigned.
5. Block release promotion on unexplained required-lane failures. Do not substitute retry-only success for a resolved Safari diagnosis.

Exit: actionable Safari and GPU failure evidence with bounded collection, plus a frozen baseline for every subsequent patch. Addresses F11, F13 and the process part of F18.

Phase 1 - Protect persistent progress (2-4 days)

1. Add meaningful regression cases derived from F01/F02 to `frontend.test.mjs`, including actual WASM and held/aborted IndexedDB transactions.
2. Define persistence ownership outside restorable machine state. Preserve a monotonic host operation generation and a battery-content revision.
3. Fix reset so it cannot discard dirty progress. Handle normal reset, imported battery reset and cartridge replacement explicitly.
4. Define load-state battery policy, drain pending saves, restore atomically and mark restored content dirty where appropriate.
5. Exercise menu reset, reset after failed autosave, repeated state loads, temporary storage, export and reopen. Keep existing transactional state/metadata writes.
6. Run `npm test`, `npm run validate`, then the software browser persistence matrix. Commit the source and regression evidence together.

Exit: no path promises durability without a successful transaction; the latest intended battery bytes survive reload. Dependencies: Phase 0 evidence is useful, but this phase need not wait for the Safari root cause.

Phase 2 - Make cartridge identity and selection reliable (2-4 days)

1. Choose canonical byte-order hashing and the versioned identity/metadata schema.
2. Add synthetic same-header/different-content and equivalent-byte-order tests before migrating storage.
3. Implement SHA-256 identity without unnecessarily duplicating a 64 MiB buffer. Separate game label, cartridge content and optional save-sharing policy.

4. Migrate legacy keys transactionally, retaining a recoverable legacy mapping and avoiding automatic ambiguous save merges.
5. Introduce a selection token before file read/decompression; make obsolete requests cancelable and unable to mutate the active session.
6. Test cached reopen, state isolation, interrupted migration, invalid newest selection and repeated large inputs. Publish backup instructions with the migration.

Exit: each library entry reopens the exact selected cartridge and the documented latest-selection policy holds. Addresses F06/F07; depends on Phase 1 persistence policy.

Phase 3 - Correct GPU ownership and lifecycle (4-8 days)

1. Specify circular intervals in byte and halfword units, including negative apron offsets and full-memory spans; add independent boundary expectations.
2. Implement splitting in core watch/stale logic and host upload/touched/readback paths. Audit CPU write exclusion lists and texture/VI callers for consistent units.
3. Add a deterministic real-GPU wrapped framebuffer/depth fixture. Verify CPU load, partial store and DMA interactions before acknowledging synchronization.
4. Add frontend scale-request generations and renderer/session ownership. Ensure stale failures cannot alter desired state.
5. Serialize scale activation/error scopes, and add owned-resource cleanup to build failure, cancellation, readback and disposal paths.
6. Test rapid scale changes during reset, state restore, software switching and deliberate device loss.
7. Run actual native and full-storage HD-at-1x comparisons. Repeat failure tests on at least one physical adapter when available.

Exit: no dropped circular range, stale request mutation, acknowledged failed barrier or unowned partial GPU allocation. Addresses F03-F05 and F11; depends on Phase 1 for state transitions.

Phase 4 - Make GPU and browser evidence mandatory (2-5 days)

1. Add a named required GPU job that provisions a known adapter, runs authored command vectors and `device_loss.mjs`, and records adapter/tool/build identities.
2. Add continuous homebrew GPU execution with save/load/reset and renderer transitions; keep native and HD-at-1x exact lanes distinct from HD display tests.
3. Expand gameplay observations to all touched targets and retained state where feasible; compare against independent expectations, not only the same implementation in another language.
4. Run the permitted commercial corpus locally with continuous GPU segments after the checkpoint-based campaign. Record initial conditions, duration, inputs and full verdicts.
5. Stabilize native Safari and add direct-file launch tests. Complete a physical iOS/iPadOS, Android Chromium and desktop Safari/Chromium matrix when those devices are available.
6. Change release validation to an explicit expected job set, including the new GPU job.

Exit: shader/coherence mutations fail required CI; unavailable coverage reports FAIL/SKIP accurately; physical and emulated devices are never conflated. Addresses F12/F13/F18. Establish the authored GPU gate early, then require it for Phase 3 acceptance.

Phase 5 - Repair bounded CPU/native conformance defects (3-6 days)

1. Fix unsigned byte assembly first and extend sanitizer inputs through actual ROM loading and peripheral reads.
2. Preserve original virtual addresses for merge-load faults and add all alignment/delay-slot cases.
3. Implement 32-bit segment privilege checks consistently in fast map, slow translation and instruction fetch paths.
4. Test invalid/missing TLB entries, ASID changes, EXL/ERL transitions and GPU restart interactions. Compare independent exception register expectations.
5. Run native ASan/UBSan and identical WASM O0/O3 vectors, then existing homebrew and commercial boot checks. Investigate any changed hashes; do not silently accept them.

Exit: F08-F10 probes cease exhibiting their reported defects, without weakening existing gates. Keep this separate from a full cache/timing/64-bit rewrite.

Phase 6 - Expand hardware compatibility deliberately (multi-week program)

1. Inventory CIC, save type, region/revision and peripheral needs for a representative permitted cartridge set.
2. Add independently checked CIC protocol vectors and implement the missing response path; reproduce an actual protected-game scenario before claiming compatibility.
3. Add save-medium overrides with backup protection and metadata validation, then EEPROM/SRAM/Flash/Pak persistence tests across power cycles.
4. Build the next independent renderer corpus: two-cycle next-pixel alpha, texel/LOD replacement, span/batch boundaries, retained depth/blender shifts, TMEM wrap/format transitions and VI interlace/filter edges.
5. Expand FPU trap/NaN/flush coverage using hardware-derived expectations where available. Track corpus provenance separately from exact-rational arithmetic tests.
6. Scope cache effects, privilege/64-bit addressing, LL/SC, CP0 timing, RSP timing and DMA scheduling as independent conformance tasks with explicit supported/unsupported behavior.

Exit: each claimed capability has independent expectations and named passing workloads; missing hardware evidence remains an explicit limitation. Addresses F16 and residual renderer/core accuracy coverage.

Phase 7 - Reduce memory and improve frame pacing (4-8 days initially)

1. Instrument ordered dispatch cost, readback time/bytes, main-thread long tasks, input latency, audio queue levels and peak memory across native/2x/4x.
2. Establish fixed replay baselines on at least desktop integrated/discrete GPUs and the affected mobile device. Report median, p95/p99 and maximum frame time, plus thermal/long-session conditions.
3. Remove avoidable full-payload metadata reads and duplicate import buffers; bound concurrent jobs and staged resources.
4. Partition provably independent renderer work and chunk long ordered passes while retaining feedback. Re-run all exact lanes after every structural change.
5. Consider a worker/OffscreenCanvas architecture only if measured main-thread stalls justify the complexity; preserve the standalone-file and browser fallback requirements.

6. Set scale defaults and fallback behavior from measured budgets. A failed high-resolution request must leave the last valid renderer and saves intact.

Exit: measurable improvements without new correctness differences, resource growth or audio regressions. Addresses F14/F15 and the performance portion of F17; depends on Phase 3 correctness and Phase 4 gates.

Phase 8 - Complete user-facing reliability and release hygiene (3-6 days, then ongoing)

1. Add audio initialization retry/cleanup and interruption tests, with underrun/overrun diagnostics.
2. Implement sibling library play/remove semantics; audit VoiceOver, focus, text scaling, contrast and touch targets on devices.
3. Add cross-tab cartridge ownership, storage usage, backup management and a documented upgrade/migration path.
4. Add multi-controller mapping as a separate feature with stable port assignment and reconnect behavior.
5. Resolve and record outstanding fixture/source provenance; package notices from the exact released revision and pin CI action identities where practical.
6. Publish a new immutable preview only after the intended exact revision passes the named release gates. Include source/HTML/WASM hashes, known issues, migration notes and actual tested platforms.

Exit: F17/F18 have explicit product behavior and recorded evidence; release notes accurately distinguish new fixes from untested claims.

5. Acceptance and delivery discipline

For every defect fix: preserve a minimal reproduction, add an independent assertion, implement the smallest coherent change, run the affected lane, then the required baseline. Renderer changes additionally require actual GPU execution; software parity alone is insufficient. New persistent schemas require backward migration and an export/recovery test. Keep observed failures in evidence instead of adjusting thresholds or dropping comparisons.

Suggested commit boundaries are persistence/reset, persistence/state-restore, identity migration, import selection, circular GPU ownership, scale lifecycle, GPU cleanup/error observation, required GPU CI, native byte assembly, CPU fault addresses, privilege checks, and Safari diagnostics. Avoid combining unrelated correctness changes with optimization. Pin the source SHA in each result file. Rebaseline only after explaining every changed result.

Release readiness is a conjunction: exact source passes the named required jobs; battery migration is verified; mandatory native Safari failures are resolved; GPU boundary/device-loss lanes pass; required provenance records and notices are correct. Physical-device performance and full-game compatibility are separately reported evidence, not implied by release success.

6. Reproduction guide and accompanying files

Run from the repository root with the existing build prerequisites. The probes print observations of defects rather than constitute a passing product test suite. UBSan cases intentionally return nonzero on the audited revision.

```
npm test
cc -O1 -g -ffp-contract=off -fsanitize=undefined -fno-sanitize-recover=all \
  reports/audit-2026-10-10/probes/core.c -lm -o out/audit-core
out/audit-core
out/audit-core rom-ub
out/audit-core bus-ub
node reports/audit-2026-10-10/probes/frontend.mjs
```

frontend.mjs reuses the existing test environment and production frontend/GPU functions. Persistence/identity probes instantiate the built real WASM; DOM and IndexedDB behavior is mocked. GPU probes mock the device or range sink. Native core.c includes the actual core. Both approaches deliberately minimize unrelated machinery. They demonstrate the stated local behavior; actual-device and full browser acceptance remains part of the implementation plan.

Files: evidence.json records source/CI/memory metadata; frontend-observations.json and core-observations.txt retain probe output; rom-ub.txt and bus-ub.txt retain sanitizer diagnostics. probes/ contains the reproducible inputs. No commercial cartridge data is included.

7. Source references

Primary project reference: [audited source tree](#). Function names and line anchors above refer to that revision; later edits may move them. The principal files examined were src/{api,bus,cpu,gpu,rdp,rsp,vi}.c, src/rdp_pixel.h, src/web/{app,gpu}.js, RDP/VI WGSL, UI markup, frontend/GPU/core test harnesses, browser/Safari runners, build scripts and both GitHub workflows.

[S1] NEC/MIPS, VR4300/VR4305/VR4310 User's Manual, U10504EJ7V0UMJ1, sections 5.2 and 6.4. Official manufacturer-authored manual hosted by an archive: <https://datasheets.chipdb.org/NEC/Vr-Series/Vr43xx/U10504EJ7V0UMJ1.pdf> . Used for address-space privilege and exception-address expectations, not as proof that all instructions were exhaustively audited.

[S2] W3C GPU for the Web Working Group, WebGPU, error handling and task-source model: <https://www.w3.org/TR/webgpu/> . Object validation and error delivery may be asynchronous; the report's specific missing-handler and completion-counter observations come from Photon64 source.

Historical project references: VALIDATION_REPORT.md, RENDERER_ACCURACY_IMPLEMENTATION.md, ACCURACY_INVESTIGATION.md, IMPLEMENTATION_PLAN.md, UI_COMPATIBILITY.md, THIRD_PARTY.md. Their successful comparisons and unresolved limits remain scoped to their recorded revisions and workloads.